INTER IIT TECH MEET

Problem Statements

Seven problems, one theme: make a machine understand its situation and act on it, under constraints that do not go away when the demo is over.

SCAN TO JOIN THE WHATSAPP GROUP

CONTENTS

01 **Drone Fleet Management for Timed Deliveries** HARD — PAGE 4

Systems & optimization — ten drones, one server, deadlines to meet, batteries to watch and not enough charging pads. Repository: Team-Deimos-IIT-Mandi/Robotic-PS

02 **Semantic Mapping of an Indoor Environment** HARD — PAGE 8

Perception — detect objects from an RGB-D stream, store their coordinates, and do it on edge hardware with no pre-built map. Repository: Team-Deimos-IIT-Mandi/Robotic-PS

03 **Dynamic SLAM in Non-Static Environments** HARD — PAGE 11

Perception & mapping — detect and reject dynamic obstacles (pedestrians, moving objects) to construct clean static maps without ghosting artifacts. Video: demo_dynamic.webm

04 **Reading the Field: VLA against World Action Models** MEDIUM — PAGE 14

Research & reading — no code. Read the papers on general-purpose robotics, pick a side on VLA versus WAM, and defend it.

05 **Hardware Abstraction Driver: CAN and UART** MEDIUM — PAGE 16

Embedded & systems — write the layer between ROS 2 and the electronics: raw CAN frames out, encoder and IMU telemetry in, nothing blocking.

06 **Kinodynamic Planning for an Ackermann Vehicle** MEDIUM — PAGE 19

Planning & control — drive a car that cannot move sideways into a parking slot, over a TCP interface, in C++.

07 **Reliable Hardware Discovery on a Deployed Robot** MEDIUM — PAGE 22

Systems & embedded — Linux hands out different port names every reboot. Find out how deep that goes. Joint software and electrical, report only.

READ THIS FIRST

How to Approach These Problems

Every problem in this booklet can be made to look solved. A framework will run, a demo will play, and nothing will have been understood. That is not what we are looking for, and it is usually obvious within the first two minutes of questions.

WHAT WE ARE NOT TESTING

Whether you can install ROS 2, get a simulator running, or wire a library into a project. Those are setup tasks. They take time, they are worth learning, and they carry almost no marks. A team that spends three weeks fighting a toolchain and one week on the actual problem has answered the wrong question.

WHAT WE ARE TESTING

- **How many edge cases you can find.** The happy path is the easy quarter of every one of these problems. What happens when the sensor returns nothing, the deadline cannot be met, the device disconnects mid-command, or two things arrive at once — that is where the work is, and where the marks are. Bring us the list of cases you thought of. Finding one we did not list is worth more than handling five we did.
- **How well you understand what you built.** Every line in your repository is yours to defend. Why this value and not a larger one. What breaks if this assumption fails. Where your design is still unsafe. “It works” is not an explanation.
- **How you approached the problem.** What you tried first, why it failed, what you changed, and what you deliberately chose not to do. The path matters as much as the destination — we would rather read an honest account of three attempts than a clean write-up of one.

A SENSIBLE ORDER OF WORK

Get the simplest possible version running end to end before you make any part of it good. A slow, ugly, complete pipeline gives you a baseline to measure against and tells you early which part is actually hard. Optimising a component before the system runs is time spent on the wrong thing. Then attack your own system: break it deliberately, write down what happened, and fix what matters.

Where a problem leaves a choice open — an algorithm, a model, a bus, a reference frame — that is deliberate. We are not withholding an answer. We want to see which one you pick, what you compared it against, and whether you can defend it.

USING LLMS

LLMs and AI coding tools are allowed on every problem here. We are not going to pretend they do not exist, and we are not testing whether you can type faster than an autocomplete. Use them.

One condition: **you own everything you submit.** If it is in your repository, you know why it is there and what happens if it changes. Expect questions on any line, without warning — why that threshold, why that data structure, what your code does when this input is malformed. “The AI wrote that part” is not an answer, and it becomes clear almost immediately. A team that built something simpler and can defend every line of it will beat a team with a better demo they cannot explain.

HOW YOU SUBMIT

One GitHub repository link, and one demo video. There are no slides. The video is how you present your work, so edit it properly: show the system actually running, cut the dead time, caption what we are looking at, and leave a failure case in. A raw unedited screen recording with no explanation does not count as a submission. Everything else a problem asks for — notes, logs, results — goes in the repository.

HARD

Drone Fleet Management for Timed Deliveries

Ten drones, one central server, and a stream of packages that each have to reach a location before a deadline. Deciding *which drone goes where, and when* is the whole problem — and it has to hold up when batteries age and every charging pad is busy.

FLEET	ARCHITECTURE	EACH PACKAGE	CHARGING	REPOSITORY
10 drones	One server, drones as clients	Location, weight, deadline	Limited pads, shared	Team-Deimos-IIT-Mandi/Robotic-PS

1 The Problem

A delivery company has ten drones and a lot of packages. Each package has a weight, a drop location and a time it must arrive by. A central server tracks every drone and decides who flies where.

One delivery is easy. Ten drones, a queue of orders, batteries that drain and slowly wear out, and a handful of shared charging pads is a different problem — and it is the one you are solving. Send the wrong drone and you burn range you needed later. Wait too long for a better option and you miss the deadline. Send everything at once and the whole fleet arrives at the pads together, and nothing flies for the next forty minutes.

2 What You Build

A central server. It holds fleet state, accepts delivery requests, and assigns them. Every decision about who flies where is made here.

Drone clients. Each drone connects to the server, reports position, battery and payload, accepts the jobs it is given, and flies them.

A fleet portal. A live view: where each drone is, what it carries, what it has been assigned, battery and health, which deliveries are on time and which are at risk, and which pads are occupied.

Portal
A live view is not decoration. It is how you show your assignment logic was sensible.

3 The Fleet You Are Given

Start from these numbers. They define the problem. If you change one, say so and justify it.

PARAMETER	VALUE	NOTE
Drones	10	Identical to begin with. See section 7.
Max payload	2.5 kg	A drone cannot lift more. Package weights run 0.2–2.5 kg.
Cruise speed	12 m/s	Falls as payload rises — model it.
Endurance	~25 min at full charge	At full payload. Longer when flying light or empty.
Charge time	~40 min, empty to full	A drone on a pad is out of the fleet.
Battery degradation	−0.05% usable capacity per full cycle	Permanent. A drone at cycle 400 no longer has its day-one range.
Base station	1	All drones launch from and return to it. See section 7 for the multi-base extension.
Charging pads	3	Three pads for ten drones.
Deadlines	10–45 min from request	Requests arrive continuously, not in a batch.

Payload matters twice over: a drone cannot carry more than its limit, and a heavy package drains the battery faster and slows the drone down — so a job that fits on paper can still be the wrong job for that drone.

4 The Decision That Matters

A new request arrives. Somewhere in your fleet a drone is already in the air, heading roughly that way. What do you do?

```
new request → 1.9 kg to drop point D, deadline in 14 min

  drone 3  airborne, passes near D in 5 min, battery 41%, already carrying 1.2 kg
  drone 7  idle at base, 9 min to D, battery 100%, capacity down 4% from age
  drone 2  charging, ready in 6 min, then 7 min to D

assign to whom? or hold the request and wait?
```

Every option costs something. Diverting the airborne drone saves a takeoff but risks its current delivery, and it may not have the spare payload. The idle drone is safe but burns a full charge on one package. Waiting for the charging drone might make the deadline and save both — or miss it by thirty seconds.

Your server makes that call, defensibly, every time. It is balancing at least four things:

- **Deadlines.** A late delivery is a failure, not a small penalty.
- **Energy and payload.** Battery spent is battery unavailable later, and weight makes every leg cost more.
- **Waiting.** Holding a request for a better assignment is allowed and sometimes correct — but a request held too long can no longer be completed at all.
- **Balance across the fleet.** Work should be spread sensibly. A schedule that runs three drones into the ground while seven sit idle looks fine on delivery numbers and quietly wears out a third of your fleet.

We are not prescribing an algorithm. Choosing one, and being able to say why it beats the alternatives you tried, is the task. What we do require is that your server can explain any single decision afterwards: why this drone, at this moment, and not the others.

5 Charging

Three pads, ten drones. A pad serves one drone at a time, and a drone on a pad is out of the fleet until it comes off.

Questions your design has to answer: who gets a pad when several drones are low at once? Does a drone reserve one before it starts flying home, and what happens to that reservation if it is diverted or arrives late? Do you send a drone to charge at 30% because the pads are free now, or keep it working and risk finding all three busy at 12%?

> **THE RULE THAT MATTERS**
>
> A drone must never be assigned a job it cannot finish, including the flight back to a pad, using its *current degraded* capacity rather than its nameplate figure. Running out of battery in the air is a crash, and a fleet manager that allows one has failed regardless of its delivery numbers.

6 Edge Cases

A large part of the marks. A system that works when everything goes right is not a fleet manager. We want the cases you thought about, what your system does in each, and how you tested it. These we already expect you to have covered — they are not the full list:

- A deadline no drone can meet. Does the server accept the request and fail, or reject it up front?
- A package heavier than any available drone can carry.
- Every pad is occupied and a drone needs one now.
- An urgent request arrives with all ten drones committed.
- Two requests, one drone that could serve either, and both deadlines cannot be met.
- A drone is diverted after it had already reserved a pad.
- A delivery is reassigned mid-flight while the first drone is still carrying the package.
- A drone reports a battery level or position that cannot be true.
- Wind or a slow leg turns an on-time delivery into a late one. When do you notice, and what then?

Finding cases we have not listed scores higher than handling the ones we have.

7 Extensions BONUS

Three optional directions, in rough order of value. Attempt them *only* once the base system is stable — a broken extension on a weak base scores lower than a solid base alone.

A — A FLEET OF DIFFERENT DRONES

Real fleets are not ten identical airframes. Make them differ — a heavy-lift drone that carries 5 kg but flies slowly and drains fast, a light fast one capped at 1 kg, an older unit with 20% of its capacity already worn away. The question sharpens from *which drone is free* to *which drone is right for this job*: the fast one cannot lift a 4 kg package, and sending your only heavy-lift drone across town with 300 g leaves the next heavy package with nothing to fly on. Define your parameter set, state it, and show your logic choosing sensibly under it.

B — COMMUNICATION FAILURES AND RECOVERY

Links drop. A drone goes quiet mid-flight and the server must decide what that means without being able to ask. Both sides need defined behaviour. On the drone: how long does it keep flying its last instruction before deciding it is on its own, and what then — continue, hold, or return — while staying inside the range that gets it home. On the server: when is a silent drone declared lost, what happens to its package, and how do you avoid giving the same delivery to two drones?

One rule if you attempt this: a drone that reconnects must **not** resume its old command. It re-identifies, reports position and payload, and waits for a fresh assignment. Think hardest about the drone you declared lost, whose package you reassigned, reconnecting while still carrying it.

C — MULTIPLE BASE STATIONS

Replace the single base with three, two pads each. The charging question gains a second half: not just who gets a pad, but *which base* they fly to. The nearest may be full while a further one sits empty, and the extra leg costs range the drone may not have.

8 Simulation

Use whatever you like — PX4 or ArduPilot SITL, Pixhawk, Aerostack, Gazebo, or any framework you are comfortable with. We are marking the fleet logic, not the flight stack.

IF THE SIMULATOR FIGHTS YOU, WALK AWAY FROM IT

If you cannot get a drone simulator running — and with ten of them, that happens — build your own 2D simulation with OpenCV and show the fleet moving on a map. That is completely acceptable and costs you nothing. A clean OpenCV visualisation with sound assignment logic beats a broken SITL setup with no logic behind it. Do not spend three weeks on the simulator and one week on the actual problem.

9 Deliverables

ITEM	WHAT IT CONTAINS
GitHub repository	https://github.com/Team-Deimos-IIT-Mandi/Robotic-PS — Server, drone clients and portal, with a README that lets us clone it and run the whole thing.
Demo video	An edited recording of a full run — requests arriving, drones assigned, batteries draining, pads filling — including a run that goes wrong.
Algorithm note	Your assignment and charging algorithms, why you chose them over what else you tried, and where they break down.
Edge case document	Every case you identified, what your system does, and how you tested it.
Results	Deliveries on time and missed, distance and energy used, idle time, pad utilisation, and how evenly work was spread.

10 What We Judge On

Assignment quality	Deadlines met, energy and payload used sensibly, and whether a specific decision holds up when we ask about it.
Edge case coverage	

	How many you found, and whether the system genuinely handles them rather than listing them.
Charging strategy	Pads used sensibly under contention, and no drone ever stranded.
Balance	Work spread across the fleet rather than concentrated on a few airframes.
Portal	Does it show enough to understand what the fleet is doing, and why.
Extensions *(bonus)*	Any of section 7 done well, on top of a base system that already works.

The first two carry the most weight. A fleet that delivers slightly less efficiently but never strands a drone and never accepts an impossible job beats a faster one that falls apart the first time something unexpected happens.

> **QUESTIONS YOU SHOULD EXPECT**
>
> Why your server sent drone 4 and not drone 9 at 03:12 in your own demo. What your cost function actually weighs, and what changes if you double one term. What happens if two requests arrive in the same millisecond. Why your charging threshold is that number, and whether it changes as a battery ages. What your system does when a drone reports a battery level that cannot be true.

HARD

Semantic Mapping of an Indoor Environment

Build a vision pipeline that detects objects in a house, works out where each one is, and stores those coordinates. It has to run on edge hardware, and it cannot use a pre-built map.

SIMULATOR	ROBOT	SENSOR	TARGET HARDWARE	REPOSITORY
Gazebo — small house world	TurtleBot 3 or 4	RGB-D camera	Jetson Nano / RPi	Team-Deimos-IIT-Mandi/Robotic-PS

1 The Problem

A robot with a depth camera is placed in a house it has no map of. It can see the room, but it does not know that the large flat thing in front of it is a bed, and it has no way to come back to that bed later.

People give robots instructions in terms of objects, not numbers — "go to the sofa", not "go to 2.4, 1.1". For the robot to act on that, it needs a stored list of objects and their positions. That list is a **semantic map**, and building it is your task.

2 What You Build

A vision pipeline that runs on a TurtleBot in the Gazebo small house world. It takes the RGB and depth stream as input. It outputs a stored list of objects.

```
{ "id": 7, "label": "coffee_table",
  "position": [x, y, z], // base point, in your chosen world frame
  "confidence": 0.87, "extent": [w, d, h] // extent optional }
```

Three requirements:

Why masks
A box around a bed also contains floor, wall and half a nightstand.

Segment, do not just box. You need the object's boundary, not a rectangle around it. A bounding box includes background pixels, and those corrupt the depth values you read from it. A pixel mask gives you only the depth that belongs to the object.

Label the object. Bed, sofa, coffee table, dustbin, chair, TV, fridge, door, and so on. Cover as many classes as your pipeline handles reliably. More classes is better, but wrong labels are worse than fewer labels.

Base point
The centre of a tall shelf sits in mid-air. A robot cannot drive to it.

Store its position. Convert the object's pixels into one 3D coordinate in a fixed world frame and save it. Use the **base point** — where the object meets the floor — because that is the point a navigation stack can drive to.

WHAT WORKING LOOKS LIKE

After a run, one command prints your stored map: *"11 objects — sofa (1.2, 3.4), dustbin (−0.8, 2.1), bed (4.5, 0.9)…"*. Send the robot to any entry in that list and it arrives at the right place.

3 Rules

NON-NEGOTIABLE

- **No pre-built map.** You cannot load a map saved from an earlier run, and you cannot read object positions out of the Gazebo world file. The robot starts with nothing, every run.
- **Peak RAM stays under 8 GB** for the whole pipeline. Measure it and report it.
- **It must run on Jetson Nano or Raspberry Pi class hardware.** Develop on your laptop, but every model you pick must be one that can actually be deployed on that board. A pipeline that only runs on a desktop GPU does not count, however accurate it is.
- **You must be able to explain everything you submit.** See *Read This First* at the front of this booklet.

The first rule is the core of the problem. Knowing where things are in advance makes this easy. Building that reference from nothing is the actual work.

4 The Coordinate Problem

Your camera gives you a position in the **camera frame**: “dustbin, 1.8 m ahead, 0.3 m to the left”. The robot then moves, and that number no longer points at the dustbin. Camera-frame coordinates cannot be stored.

You need a fixed reference frame that holds for the whole run. **We are not telling you what it should be.** Picking it, building it and defending it is part of the problem.

> **QUESTIONS YOUR DESIGN HAS TO ANSWER**
>
> - **Where is your origin?** The start pose is one option. Is it enough? What breaks if the robot is moved and restarted somewhere else?
> - **How do you track the robot's pose?** Whatever you choose, say what it costs you in compute and how much it drifts.
> - **What happens as drift builds up?** After a full loop of the house, is the sofa still where you recorded it? How far off is it? Do you correct it, or measure and report the error?
> - **What if you see the same object again?** You will pass the same sofa four times. Do you end up with four sofas in your map?
> - **Which single point represents the object?** You have thousands of depth values inside a mask. How do you reduce them to one coordinate, and then to a point on the floor?

There is no single correct answer. A simple approach you fully understand, whose failure cases you can name, scores better than a complex one you copied.

5 Running on Small Hardware

The constraint is the point of this problem, and a large share of the marks sits here. Wiring a large model to a camera is easy. Making it small and fast while keeping it correct is the work.

We prescribe no models, no runtimes and no techniques. Finding them is your job, and it is left open on purpose — two teams that solve this well should not end up with the same pipeline. Whatever you pick, be ready to say why you picked it over what else you tried, and what it cost you.

> **MEASURE AND REPORT**
>
> Peak RAM · end-to-end FPS and per-stage latency · model size on disk · accuracy. Keep a log from day one: every change you made, the numbers before and after, and one line on whether you kept it. A row reading *“4 FPS → 19 FPS, mask IoU −3%”* tells us more than any final number, and that log is the strongest thing you can show a judge.

6 Optional Extension BONUS

You may use a VLM or VLA. Stopping at a clean, fast, accurate map is a complete submission. If you want to go further: pass your stored map and a plain instruction to the model, and have it return a structured command the robot can execute. Send that to the navigation stack and let the robot drive there.

```
Input: "go and check the dustbin" + your stored object map
Output: { "action": "navigate_to", "target": "dustbin",
          "coordinates": [-0.82, 2.14, 0.0] }
```

The output must be parseable structure, not prose — validate it, and handle the case where the model returns nonsense. Respect the same compute budget if you claim it runs on the edge; if you offload it, say so. Do not weaken the core pipeline for this. A broken bonus on a weak base scores lower than a strong base alone.

7 Deliverables

ITEM	WHAT IT CONTAINS
GitHub repository	https://github.com/Team-Deimos-IIT-Mandi/Robotic-PS — The working pipeline, and a README that lets us clone it and reproduce your run from scratch.
Semantic map output	A JSON or YAML object list — label, 3D coordinate, confidence — from a real run.
Demo video	An edited recording of the robot exploring, with detections and the map building live, plus a failure case. See the note below.

Benchmark report	The measurements and the change log from section 5.
Design note	A few pages: your reference frame and why, your model choices and why, and the failure modes you know about.

8 What We Judge On

CRITERION	
Optimization & edge readiness	Measured gains, real deployability on the target board, memory discipline, and a clear record of your trade-offs.
Pipeline design	Clean, justified architecture. A sound reference frame. Repeat detections handled properly.
Mapping accuracy	Correct labels and coordinates, sensible base points, class coverage, no duplicate objects.
Robustness	Survives a different start pose, route, lighting and occlusion. Degrades instead of failing outright.
Understanding & defence	Can you explain every decision under questioning.
Optional extension	Instruction to structured command, grounded in your own map, ending in real robot motion.

The first two carry the most weight. A team at 92% accuracy and 2 FPS on a desktop GPU loses to a team at 84% and 15 FPS on a Nano. That ordering is deliberate.

> **QUESTIONS YOU SHOULD EXPECT**
>
> Why this model and not the other three you tried. Why this threshold and not a higher one, and what changed when you moved it. Why you reduced the mask that way. What your code does when the depth sensor returns zeros. What happens the third time the robot passes the same sofa.

ONE WARNING

The most common way teams lose this: three weeks spent raising accuracy on a laptop GPU, then finding in the last week that none of it fits the target board. Test against your constraints from week one.

PROBLEM STATEMENT 03 / PERCEPTION & MAPPING

HARD

Dynamic SLAM in Non-Static Environments

Standard SLAM assumes the world is frozen in time. When people walk past or doors open, moving points get baked into the map — leaving ghost obstacles, blurry occupancy grids, and drifted trajectories. This task is to investigate how to map when the world moves around you.

TYPE	DOMAIN	ENVIRONMENT	DEMO VIDEO	REPOSITORY
Open-ended investigation	LiDAR / RGB-D SLAM	Gazebo / Isaac Sim with dynamic actors	demo_dynamic.webm	Team-Deimos-IIT-Mandi/Robotic-PS

1 What Happened

Most simultaneous localization and mapping (SLAM) frameworks — whether classical scan-matchers like Cartographer and GMapping, or visual-depth pipelines like ORB-SLAM3 and RTAB-Map — rely on a core assumption: *the environment is static*. Every point returned by a 3D LiDAR or depth camera is treated as an immutable physical surface.

When a person walks past a scanning robot, their surface points are integrated into the global occupancy grid or point cloud map. As they move across several frames, the system leaves a trailing tail of false obstacles ("ghosting"). Subsequent navigation queries see a path blocked by phantom obstacles that no longer exist, while scan-matching algorithms drift because they attempt to align new sensor frames against moving dynamic landmarks.

```
static assumption  P(world) = fixed → clean wall geometries & crisp occupancy grid
dynamic reality    pedestrians / vehicles → ghost trails, blurry grids, localization drift
```

Clearing these ghosts with standard raytracing is slow, computationally expensive on edge hardware, and often fails when dynamic objects stop moving temporarily.

2 What This Task Actually Is

> READ THIS FIRST
>
> Applying a naive bounding box detector or cranking up costmap inflation parameters takes a few hours and is **not** what we are asking for. We want to see **how deeply you analyze the dynamic SLAM problem** — geometric residual filtering, semantic feature masking, optical flow variance, spatiotemporal occupancy decay, and robust optimization cost functions. The investigation is the deliverable; a benchmarked prototype pipeline is your evidence.

Your goal is to investigate, implement, and benchmark strategies that isolate and reject dynamic obstacles before or during map optimization. The robot must produce a crisp, static background map while accurately tracking localized dynamic obstacles in real time.

LITERATURE REVIEW & SYNTHESIS

Numerous research papers explore dynamic SLAM — from visual/depth approaches like DynaSLAM, DS-SLAM, and Mask-SLAM to LiDAR removal techniques like ERASOR, Removert, and SuMa++. We expect teams to review published literature, analyze key algorithms against real-time edge constraints, and synthesize a state-of-the-art pipeline that yields the best balance of static map crispness, low localization drift, and real-time performance.

3 Ground to Cover

Geometric & Kinematic Methods: Scan-to-scan residual checks, ICP/NDT outlier rejection, range-image differential analysis, point cloud velocity estimation, spatio-temporal voxel decay.

Semantic & Learning-Based Masking: Real-time semantic segmentation (e.g. YOLO/MobileNet maskers), optical flow dynamic keypoint rejection, dynamic feature filtering in visual/RGB-D SLAM front-ends.

Map Representation & Filtering: OctoMap / VoxelGrid probability updating, ray-casting clearing vs Bayesian belief updates, static vs dynamic layer separation in ROS 2 costmaps.

```
sensor stream → dynamic detection & segmentation → feature/point filtering
→ robust pose optimization → temporal occupancy decay → CRISP STATIC MAP
```

4 Questions We Want Answered

The core tension
A person stops walking for 30 seconds. Is it now a static obstacle or still dynamic?

Static vs Dynamic Split	How does your system distinguish between moving objects (pedestrians), temporarily stationary objects (a chair moved across the room), and true static structures (walls)?
Localization Impact	How much does dynamic clutter degrade pose estimation accuracy (ATE/RPE RMSE), and how much drift reduction is achieved by masking dynamic features?
Ghost Removal Time	When a dynamic object traverses a region, how many frames or seconds does it take for the occupancy grid to clear the ghost trails?
Edge Real-Time Performance	What is the computational overhead (latency in ms, FPS drop, peak RAM) of adding dynamic filtering to a standard SLAM pipeline on Jetson / RPi target hardware?
False Negatives	What happens if a static wall or structural pillar is misclassified as dynamic and filtered out? How does the map recover?
Low-Texture / Multi-Object	How does the filtering hold up in crowded environments with multiple intersecting dynamic trajectories?

5 Break It on Purpose

Test your pipeline under adverse dynamic conditions in Gazebo or Isaac Sim. Do not evaluate only with a single isolated pedestrian.

- **The Stationary Person:** An actor walks into the room, sits on a chair for 45 seconds, then walks away. Does the chair or person become permanent?
- **Crowded Chokepoint:** 5+ dynamic actors walking continuously through a narrow corridor during mapping.
- **Fast vs Slow Motion:** High-speed objects (drones/rc cars) vs extremely slow-moving objects (crawling robots/turtles).
- **Occlusion & Shadowing:** Dynamic actors passing directly between the sensor and static landmark features.

Record map entropy, trajectory ATE (Absolute Trajectory Error), occupancy grid IoU against ground truth static environment, and processing frame rate for each scenario.

6 Deliverables

ITEM	WHAT IT CONTAINS
GitHub repository	https://github.com/Team-Deimos-IIT-Mandi/Robotic-PS — Pipeline source code, benchmark scripts, and configuration.
Demo video	https://github.com/Team-Deimos-IIT-Mandi/Robotic-PS/blob/main/demo_dynamic.webm — Video demonstrating live dynamic SLAM mapping vs static baseline.
Benchmark report	Quantitative evaluation metrics (ATE/RPE, map quality, ghosting decay times, latency).
Design note	Detailed analysis of dynamic filtering algorithms, failure modes, and edge deployment feasibility.

7 Using LLMs

Allowed — use them. One condition: you must understand every equation, cost function term, and filtering heuristic in your submission. Ensure you can explain why specific dynamic points were misclassified and how your residual thresholds behave under sensor noise.

THE CORE RESEARCH QUESTION

How can a mobile robot reliably construct a clean, accurate static map and maintain drift-free localization in a dynamic, highly populated environment with minimal latency and zero prior map knowledge?

MEDIUM

Reading the Field: VLA Models against World Action Models

No code in this one. Read the research on general-purpose robotics, work out where the field actually disagrees with itself, and tell us which side you think is right — and why.

TYPE	TO READ	TOPIC	OUTPUT
Reading and opinion	Surveys, plus 3 research papers	VLA models vs World Action Models	Notes and a written opinion

1 Why This Exists

Inter-IIT High Prep and Mid Prep problem statements have been mostly research-based for the last few years. The technical, implementation-heavy ones are covered by the other practice problems in this booklet. This one covers the other half.

The gap we keep seeing, particularly in second years and a fair number of third years, is the ability to read a research paper, tell a weak one from a strong one, and find the right papers for a problem in the first place. If you have not worked on a research-based problem before, that skill is not obvious and it does not appear on its own. This problem statement exists to build it.

2 The Question

Some of the most interesting work in AI and robotics right now is in **general-purpose robotics**: one system that can be told to do many different things, rather than one trained for a single task. Two big ideas compete in that space.

The split
Google DeepMind and Physical Intelligence are betting on VLAs. NVIDIA is moving towards WAMs. Plenty of researchers argue for a hybrid.

Vision-Language-Action (VLA) models and **World Action Models (WAMs)**. The field has not settled which is the better foundation, and serious labs are betting in opposite directions.

Read into it, and then answer the question we care about:

> **WHAT WE ARE ASKING**
>
> Which approach do *you* think is more suitable for general-purpose robotics, and why? Your answer needs to be your own, argued from what you read. It may well be wrong. That is completely fine — this is not settled in academia either.

3 What to Read

Start with survey papers. They will give you the shape of the field and point you at the work worth reading properly.

Then read **three research papers** in full. One condition on the choice: the three must **not** be on very similar topics. Three papers arguing the same point from the same angle teach you one thing. Pick papers that cover different aspects of the problem, and if two of your three feel interchangeable, replace one.

FINDING PAPERS WORTH YOUR TIME

Use Google Scholar and Google Scholar Labs. Two rough filters while you search: papers with a significant number of citations, and papers coming out of well-known companies, universities and research institutions. Neither filter is perfect, but both beat picking whatever appears first.

Make proper notes on each paper. We will ask questions based on them during evaluation, so write them for yourself rather than for us — what the paper claims, what it actually demonstrates, what you did not follow, and where you think it is weak.

4 If a Paper Goes Over Your Head

Some of this material is genuinely advanced. If a paper leans on concepts you have not met yet, transformers being the obvious example, or if the maths is beyond where you are right now, **skip those parts**. You are not being marked on whether you can follow every equation.

What we are checking is whether you can research a problem properly and form your own understanding and opinion from it. That skill is what makes a High Prep or Mid Prep problem statement survivable. If you are interested in robotics, the reading is worth doing for its own sake regardless of how this is marked.

5 Using LLMs on This One

The booklet-wide rule applies here too, with one addition that matters more on a reading task than anywhere else.

> **THE LINE**
>
> You are free to use an LLM as an assistant, and to use one to understand topics or concepts you are unfamiliar with. **Do not paste this question into a Pro model and submit what comes back.** The point of this problem is not to test how well you can generate an answer. It is to test how well you can research, understand, compare, and form an opinion about an open research problem. An answer that reads like a model wrote it will be obvious, and the follow-up questions will make it more so.

6 What to Hand In

ITEM	WHAT IT CONTAINS
Reading notes	One set per paper. What it claims, what it shows, what you could not follow, and what you think is weak about it.
Your opinion	A short written argument for VLA, WAM, or a hybrid, built on the papers you read. State what would change your mind.
Paper list	The surveys and the three papers, with a line each on why you picked that one and not something adjacent.

Submit as a GitHub repository link, same as the other problems. No demo video is needed for this one.

7 What We Judge On

CRITERION	
Paper selection	Did you find work that is actually relevant and actually good, and do your three cover genuinely different ground?
Depth of reading	Do the notes show you engaged with the papers, including the parts that were hard?
Quality of the argument	Is the opinion yours, is it supported by what you read, and do you know where it is weak?
Honesty	Saying “I did not understand this section” costs you nothing. Pretending you did costs you a lot.
Understanding & defence	Can you answer questions on any paper you listed.

> **QUESTIONS YOU SHOULD EXPECT**
>
> What this paper's main claim is, in your own words. What experiment supports it, and whether that experiment actually shows what they say it does. Why you chose this paper over the three others on the same topic. Which of your three papers you trust least, and why. What would have to be true for you to switch sides on the VLA and WAM question.

PROBLEM STATEMENT 05 / EMBEDDED & SYSTEMS

MEDIUM

Hardware Abstraction Driver: CAN and UART

Write the C++ layer between ROS 2 and the vehicle's electronics: wheel commands out as raw CAN bytes, encoder frames back, an IMU stream off a serial port — and nothing that blocks the control loop.

BUSES	DEVICES	LANGUAGE	CONTROL LOOP	IMU STREAM
SocketCAN + POSIX serial	`vcan0` · `/tmp/ttyV0`	C++ (ros2_control)	100–500 Hz	100 Hz

1 The Problem

In simulation, a robot's wheels turn because a plugin says so. On a real vehicle nothing turns until some software has packed a velocity into eight bytes and put them on a CAN bus. Between the high-level controller and the motors sits a layer most people never write. That is the layer you are writing here.

You are given a firmware emulator that behaves like the vehicle's onboard microcontroller. It listens on a virtual CAN bus and a virtual serial port, acts on what you send, and streams telemetry back.

2 What You Build

A custom C++ hardware interface — a `ros2_control SystemInterface` — that sits under the standard controllers and owns both buses.

```
[ ROS 2 controllers ]  (diff drive / joint trajectory)
          |
[ your hardware interface ]  <-- the deliverable
          |
          +---> SocketCAN vcan0  ----> [ firmware emulator ] ---> rover kinematics
          +---> UART /tmp/ttyV0  ---->
```

Five stubs in `apps/candidate_template.cpp`:

FUNCTION	WHAT IT DOES
`init()`	Open and configure the non-blocking CAN socket on `vcan0`, and the termios serial port on `/tmp/ttyV0`.
`computeCanChecksum()`	8-bit XOR over payload bytes 0–6.
`sendMotorVelocity()`	Build and write the 8-byte motor command frame.
`readEncoderFeedback()`	Non-blocking read of encoder frames, checksum verified before use.
`readImuTelemetry()`	Non-blocking read and parse of NMEA IMU sentences, CRC verified before use.

3 CAN Protocol

Raw SocketCAN — `AF_CAN`, `SOCK_RAW`, `CAN_RAW` — on interface `vcan0`. Four motors, one ID each way.

Commands go out on `0x101–0x104`. **Encoder feedback** comes back on `0x201–0x204`. In both directions the ID's low nibble is the motor number.

Endianness
Float32, little-endian, at byte offset 1 — an un-aligned offset.

BYTE	TYPE	CONTENTS
0	**uint8_t**	Motor identifier — 1, 2, 3 or 4.
1–4	**float32**	Target velocity going out, measured velocity coming back. Little-endian.
5–6	**uint8_t[2]**	Reserved. Send `0x00`.
7	**uint8_t**	XOR checksum of bytes 0 through 6.

4 UART Protocol

The IMU streams ASCII NMEA sentences at 100 Hz. Port settings are 115200 baud, 8N1, raw mode.

```
$ROVER,IMU,<accel_x>,<accel_y>,<yaw>*<CRC_HEX>\n
example: $ROVER,IMU,0.120,-0.050,1.5708*3E\n
```

The CRC is two hex characters: the 8-bit XOR of every ASCII character between `$` and `*`, excluding both. For the example above, that is the XOR over `ROVER,IMU,0.120,-0.050,1.5708`.

Note
Your `/tmp/ttyV0` and the emulator's `/tmp/ttyV1` are the two ends of one virtual pair.

> **WHERE THIS ONE BITES**
>
> A non-blocking read returns whatever bytes happen to be in the buffer. That is rarely one whole sentence — you will get half of one, or one and a half. If you parse only what a single `read()` handed you, you will drop most of the stream and never see why. Hold a buffer across calls, and pull out complete sentences as they form.

5 Real-Time Rules

> **NON-NEGOTIABLE**
>
> - **Nothing blocks.** `read()` and `write()` run inside a 100–500 Hz control loop. Every CAN and serial call must return immediately when there is no data: `O_NONBLOCK` on the socket, `VMIN=0` and `VTIME=0` on the port. One blocking call stalls the entire robot, not just your driver.
> - **Verify before you trust.** Check byte 7 on every CAN frame and the CRC on every NMEA sentence *before* the value reaches wheel or sensor state. A corrupt frame is dropped, not used.
> - **No sleeping, no busy-waiting, no retry loops** inside the control cycle to make up for a read that came back empty. Empty is a normal outcome.
> - **You must be able to explain everything you submit.** See *Read This First* at the front of this booklet.

Everything above is checkable from the outside. The loop either holds its rate or it does not, and bad frames either reach the state or they do not.

6 Test and Verify

```
./setup_vcan_pty.sh                    # bring up vcan0 and the pty pair

# terminal 1 — the emulated microcontroller
python3 firmware_emulator/rover_firmware_emulator.py

# terminal 2 — build and run your driver
mkdir -p build && cd build && cmake .. && make -j4
./candidate_template vcan0 /tmp/ttyV0
```

When it is right, you get a live log of encoder feedback and IMU acceleration and yaw. When it is nearly right, you get a log that looks fine and drops a quarter of the frames — so count what you receive against what the emulator sent, rather than trusting the output on sight.

7 Deliverables

ITEM	WHAT IT CONTAINS
GitHub repository	One link. Your completed hardware interface, building cleanly against the given CMake project, with a README covering how to run it.
Demo video	An edited recording showing encoder and IMU data flowing over the virtual buses, plus a failure case. See the note below.
Evidence it works	Your own counts: frames sent, frames received, checksums failed, sentences dropped.
Loop timing	Measured cycle time for `read()` and `write()` at your control rate, with the worst case, not just the average.
Design note	A few pages: how you handle partial reads, what you do with a failed checksum, and where your driver would break on real hardware.

8 What We Judge On

CRITERION	
Correctness on the wire	Frames the emulator accepts, checksums computed right in both directions, floats decoded correctly.
Real-time behaviour	Genuinely non-blocking, holds its rate under load, no hidden waits or sleeps in the cycle.
Data integrity	Corrupt frames rejected, partial sentences handled, nothing bad reaching the state.
Robustness	Survives the emulator restarting, a bus going quiet, and malformed input, without crashing or hanging.
Code quality	Clean resource handling, no leaked descriptors, errors handled rather than ignored.
Understanding & defence	Can you explain every decision under questioning.

Correctness and real-time behaviour carry the most weight. A driver that decodes everything perfectly but blocks for two milliseconds on an empty bus has failed the part of this problem that matters.

> **QUESTIONS YOU SHOULD EXPECT**
>
> Why you configured termios the way you did, and what each flag you set actually turns off. How you get a float out of bytes 1–4, and what your method assumes about alignment. What your code does when `read()` returns `EAGAIN`. What happens if a sentence arrives split across two reads, and how you tested that it does. Why your checksum runs over bytes 0–6 and not the whole frame.

PROBLEM STATEMENT 06 / PLANNING & CONTROL

MEDIUM

Kinodynamic Planning for an Ackermann Vehicle

Write a C++ client that connects to a car simulator, plans a path the car can actually drive, and streams steering and speed commands until it reaches the goal pose without hitting anything.

SIMULATOR	INTERFACE	LANGUAGE	CONTROL RATE	SCENARIOS
C++ kinodynamic sim, 100 × 100 m grid	TCP socket, port 8091	C++17	20 Hz	4

1 The Problem

A car cannot move sideways. To shift one metre to its left it has to drive forward, turn, reverse, and straighten up. This is what makes parking hard, and it is why a path that looks fine on a map is often a path the car physically cannot follow.

Non-holonomic
Fewer ways to move than there are dimensions to move in. A car has three — x, y, heading — but only two controls.

You are given a simulator of such a car. It runs the vehicle dynamics, checks collisions, and exposes everything over a TCP socket. Your job is to write the software that drives it: read the map and the goal, plan a path the car can follow, and send the controls that keep it on that path.

2 What You Build

A C++ client. It connects to the simulator, requests the configuration and the occupancy grid, plans a collision-free path from the start pose to the goal pose, and streams control commands at 20 Hz until the goal is reached.

Two halves, and both have to work:

The planner produces a path the vehicle can actually drive — never a turn tighter than the steering limit, never the car body through an obstacle. Position alone is not enough: heading is part of the state, and so is direction of travel. Reversing is allowed, and in the parking scenarios it is required.

Goal pose, not goal point
Arriving at the right place facing the wrong way is not a park.

The controller follows that path on the real vehicle model, which will not track it perfectly. It corrects the error as it builds, and brings the car to the goal *pose* — position and heading, within tolerance.

Your logic goes in `apps/candidate_template.cpp`.

3 The Vehicle

Motion follows the Ackermann kinematic bicycle model. Your two controls are target speed v and steering angle δ.

PROPERTY	VALUE	WHAT IT MEANS FOR YOU
Wheelbase	2.5 m	Sets how sharply the car turns for a given steering angle.
Max steering angle	δ_{max} = 0.60 rad (≈ 34.3°)	A hard limit. Commands beyond it are not achievable.
Min turning radius	$R_{min} = L_w / \tan \delta_{max} \approx$ 3.7 m	The tightest arc the car can trace. Any planned curve tighter than this is undrivable.
Body size	4.0 m × 1.8 m	Collisions are checked on the oriented bounding box, so heading changes the footprint.

THE CONSEQUENCE

The car sweeps a 4.0 × 1.8 m box that rotates as it turns. A path that clears every obstacle as a line of points can still put a corner of the car into a parked vehicle. Check the body, not the centre.

4 Scenarios

Four scenarios, selected by ID when you launch the simulator. Your client should handle all of them without being rewritten for each.

ID	SCENARIO	WHAT IT TESTS
0	**Street parallel parking**	Parallel park into a tight slot between two parked cars along a curb.
1	**Parking lot bay**	Park into a perpendicular bay with tight aisles on either side.
2	**Slalom track**	Navigate an obstacle course. Continuous steering, no stopping to think.
3	**Multi-goal navigation**	Reach several goal waypoints in sequence.

5 Protocol

The simulator runs a TCP server on port `8091`. Four messages, all newline-terminated plain text.

REQUEST THE CONFIGURATION — CLIENT TO SERVER

Send `Q\n` once at startup. You get back the poses, the vehicle parameters and the map.

```
CONFIG <start_x> <start_y> <start_yaw> <goal_x> <goal_y> <goal_yaw>
       <len> <width> <wheelbase> <max_steer> <max_speed> <min_speed>
       <map_w> <map_h> <res> <orig_x> <orig_y> <cols> <rows>
GRID <num_cells> <val_0> <val_1> ... <val_N>   // 0 = free, 1 = obstacle
```

STREAM CONTROLS — CLIENT TO SERVER, 20 HZ

```
CTRL <target_speed_m_s> <target_steering_angle_rad>
e.g. CTRL 1.2 -0.25
```

TELEMETRY — SERVER TO CLIENT

```
TELEMETRY <step_count> <time_ms> <x> <y> <yaw> <v> <delta>
          <is_colliding> <is_goal_reached>
```

UPLOAD YOUR TRAJECTORY FOR VISUALISATION — OPTIONAL

```
TRAJ <x1> <y1> <yaw1> <v1>;<x2> <y2> <yaw2> <v2>;...
```

> **WORTH KNOWING BEFORE YOU START**
>
> The grid arrives once, as a flat array — you convert to world coordinates yourself using `res`, `orig_x` and `orig_y`. Telemetry is a stream, so your loop has to keep reading while it keeps sending. Miss the 20 Hz control rate and the car runs on stale commands.

6 Build and Run

You need GCC/G++ 11 or newer, CMake 3.16+, and OpenCV 4 (`libopencv-dev`).

```
cd ackermann_kinodynamic_sim
mkdir -p build && cd build
cmake .. && make -j4

./simulator_node 8091 0   # port, then scenario ID 0-3

# in a second terminal
./candidate_template 127.0.0.1 8091
```

7 Rules

NON-NEGOTIABLE

- **Respect the vehicle model.** No teleporting, no sideways motion, no steering beyond δ_{max}. If the plan needs a radius under 3.7 m, the plan is wrong.
- **No collisions.** Checked on the oriented bounding box of the full car body.
- **Reach the goal pose** — position and heading within tolerance.
- **One client, all four scenarios.** No per-scenario hard-coding, no replaying a recorded control sequence.
- **C++.** Your logic goes in `apps/candidate_template.cpp`.
- **You must be able to explain everything you submit.** See *Read This First* at the front of this booklet.

We prescribe no planner and no controller. Which algorithms you use, and how you combine them, is the problem. Whatever you pick, be ready to say why you picked it over what else you tried, and where it breaks.

8 Deliverables

ITEM	WHAT IT CONTAINS
GitHub repository	One link. Your client, building cleanly against the given CMake project, with a README covering how to run it.
Run logs	One per scenario: time to goal, collisions, final pose error, and whether the goal was reached.
Demo video	An edited recording of all four scenarios with your planned trajectory visible via `TRAJ`, plus a failure case. See the note below.
Design note	A few pages: your planner, your controller, how they connect, and the cases where your approach struggles.

9 What We Judge On

CRITERION	
Does it park	Goal reached, in pose, without collisions, across all four scenarios.
Planner & controller design	Sound handling of the non-holonomic constraint, clean separation of planning and control, sensible replanning.
Path quality	Smooth, drivable paths. Sensible clearance. Not more reverse manoeuvres than the situation needs.
Robustness	Handles all four scenarios with one codebase, and recovers when tracking drifts instead of failing outright.
Efficiency	Plans fast enough to hold 20 Hz control, and does not stall the loop when it replans.
Understanding & defence	Can you explain every decision under questioning.

QUESTIONS YOU SHOULD EXPECT

Why this planner and not the other one you tried. What your cost function penalises, and what changes if you double one term. Why the car swings wide on the third turn of the slalom. What your controller does when the tracking error grows instead of shrinking. What happens if the goal is placed inside an obstacle.

PROBLEM STATEMENT 07 / SYSTEMS & EMBEDDED

MEDIUM

Reliable Hardware Discovery on a Deployed Robot

Our robot runs a Raspberry Pi or Jetson with several ESP controllers on USB. Linux does not give them the same port names twice. This task is not to patch that — it is to find out how deep the problem goes.

TYPE	TEAMS	PLATFORM	OUTPUT
Open-ended investigation	Software + Electrical / Embedded	Raspberry Pi or Jetson, ESP controllers	Investigation, not a fix

1 What Happened

Controllers appear as `/dev/ttyUSB0`, `1`, `2`. The numbers follow the order Linux happens to detect the devices, and that order changes after a reboot, a replug, or an ESP reset.

```
expected  ttyUSB0 → LEFT_MOTOR   ttyUSB1 → RIGHT_MOTOR   ttyUSB2 → ARM
after replug  ttyUSB0 → RIGHT_MOTOR   ttyUSB1 → ARM   ttyUSB2 → LEFT_MOTOR
```

Software that assumes `ttyUSB0` is the left motor then sends a *valid* command to the wrong controller. Nothing errors. One side of the robot moves. On a machine with mass and torque that is a safety failure, not a config bug.

2 What This Task Actually Is

> **READ THIS FIRST**
>
> A script that reads `/dev/serial/by-id` and maps names takes an afternoon, and is not what we are asking for. We want to see **how far into the stack you are willing to go** — kernel, driver, udev, firmware, protocol, bus — and how well you reason about failure once you are down there. The investigation is the deliverable; a prototype is evidence, not the answer.

Two rules anchor the design. The robot identifies hardware by **what the device is**, never by the port number Linux gave it. And when identity or health cannot be established with confidence, the system **refuses to send commands** rather than guessing.

Both teams contribute. This is not a software task with electrical support, or the reverse. The failure crosses the boundary — Linux cannot identify a controller that has no identity to report, and firmware cannot fail safe if nothing upstream notices it stopped answering. Software and Electrical each own the areas below, and the report is written jointly.

3 Ground to Cover

Software: USB enumeration and detection order, the Linux device model, kernel uevents, usb-serial driver binding, udev and persistent names, hot-plug and removal, device-manager architecture, connection state machines, ROS 2 integration, diagnostics.

Electrical: ESP unique ID and role, firmware identification, protocol design and versioning, heartbeat, command timeout, watchdog, safe motor state, reset and brown-out behaviour, CAN and CAN-FD.

Trace the full path and know what each stage owns:

```
physical USB → USB subsystem → kernel device model → usb-serial driver
→ uevent → udev → /dev node → device manager → identify → verify → READY
```

Watch it happen rather than read about it — `dmesg -w`, `udevadm monitor`, `udevadm info`, `lsusb -t`, `/dev/serial/by-id/`. Plug, unplug, reset, swap, and record what the system does.

4 Questions We Want Answered

The hard one
Two ESPs behind identical USB-UART chips report identical VID/PID. What survives that?

Why it happens	What exactly decides that this device becomes `ttyUSB0`, and where in the stack?
Identity	VID/PID, USB serial, `by-id`, udev name, MCU unique ID, application handshake — which is trustworthy on a deployed robot, and which fails when?
Kernel vs userspace	What can udev alone solve, what must live in userspace, and where is the boundary?
Verification	How does software confirm the device on this connection is the one it expects — and what does it do when the answer is no?
Protocol	What must a command carry beyond a number, so a stale, duplicate, delayed or corrupted packet cannot be acted on?
Dead master	The Pi crashes mid-motion. Nothing is left to command a stop. How does the ESP stop anyway?
Recovery	How does the system get from LOST back to READY with no restart, and how long does each step take?
ROS 2	Who owns hardware state, how does a lost controller surface, and does the robot run degraded when a non-critical device dies?
Architecture	Is USB the right bus at all against CAN / CAN-FD — judged on reliability, diagnosability, latency, fault isolation and scale.

> **ONE RULE YOU MAY NOT DESIGN AROUND**
>
> Reconnection must not resume the old motion command. Communication returns, the device is re-identified, the handshake completes — and the robot stays stopped until a *fresh* command arrives.

5 Break It on Purpose

Not finished until you have attacked it yourself. Unplug mid-motion, swap two cables, swap two ESPs, reset one, pull the hub. Kill the device manager, kill the ROS 2 node, stop the heartbeat. Send malformed, stale, duplicate and delayed packets. Connect the wrong ESP, a duplicate ID, old firmware, an unsupported protocol version.

Record the same five things every time:

```
failure → how long until detected → what the robot did → safe state reached?
→ how it recovered → what the logs showed
```

Detection time is the field people forget. A fault caught in 800 ms and one caught in 80 ms are different systems.

6 What to Hand In

One final report of your findings, written by both teams, submitted as a **GitHub repository link**. Nothing else is required — no product, no polished system, and no slides. If you built something along the way, put it in the same repository and add a short edited video showing it, as evidence of what you observed.

The report should cover:

- What you investigated and what you actually observed — captured output, not paraphrase — including the approaches that did not work and why.
- Every failure you induced, with the five fields from section 5 filled in.
- Your heartbeat, command timeout, watchdog and recovery values, each with the measurement behind it rather than a round number.
- The architecture you would actually deploy, its state machine, and an honest list of what it does not handle.
- USB against CAN / CAN-FD for the deployed robot, argued on our requirements.

7 What We Judge On

Depth	How far down the stack you went, and whether you stopped at the first layer that produced a working answer.
Evidence	Claims backed by what you observed on real hardware, not by documentation you read.
Safety reasoning	Does the design refuse to act when unsure — and did you find the cases where it would still act wrongly?
Failure handling	Quality of the fault injection, and whether your numbers are justified.
Generality	Does this become an architecture the team reuses on the next robot, or a fix for this one?
Defence	Can you explain every decision, and every layer you touched, under questioning.

A team that maps ports correctly and stops there scores below a team whose system still has a gap — if they can say precisely where the gap is and why it is hard to close.

8 Using LLMs

Fair warning
This one is defended almost entirely in the viva. Depth cannot be borrowed.

Allowed — use them. One condition: you own everything you submit. Expect questions on any claim. Which process created that device node, and how do you know. What your udev rule matches on, and what happens when two devices match it. What you measured to arrive at that watchdog value. What the robot does in the two seconds after the Pi dies. Where your design is still unsafe.

> **THE QUESTION BEHIND ALL OF THIS**
>
> How do you design a Linux-based robot in which controllers are discovered, identified, verified, monitored, lost and recovered automatically — with no restart — even when ports change, devices reboot, communication fails, or hardware is swapped?